

# Lab Assignment-1

Understanding Algorithm Efficiency through Analysis, Recurrences, and Sorting

## Task 1: Algorithm Growth Observation

### Aim:

To observe how execution time grows for different time complexities.

### Input Sizes Used:

$n = 10, 100, 500, 1000$

### Algorithms Implemented:

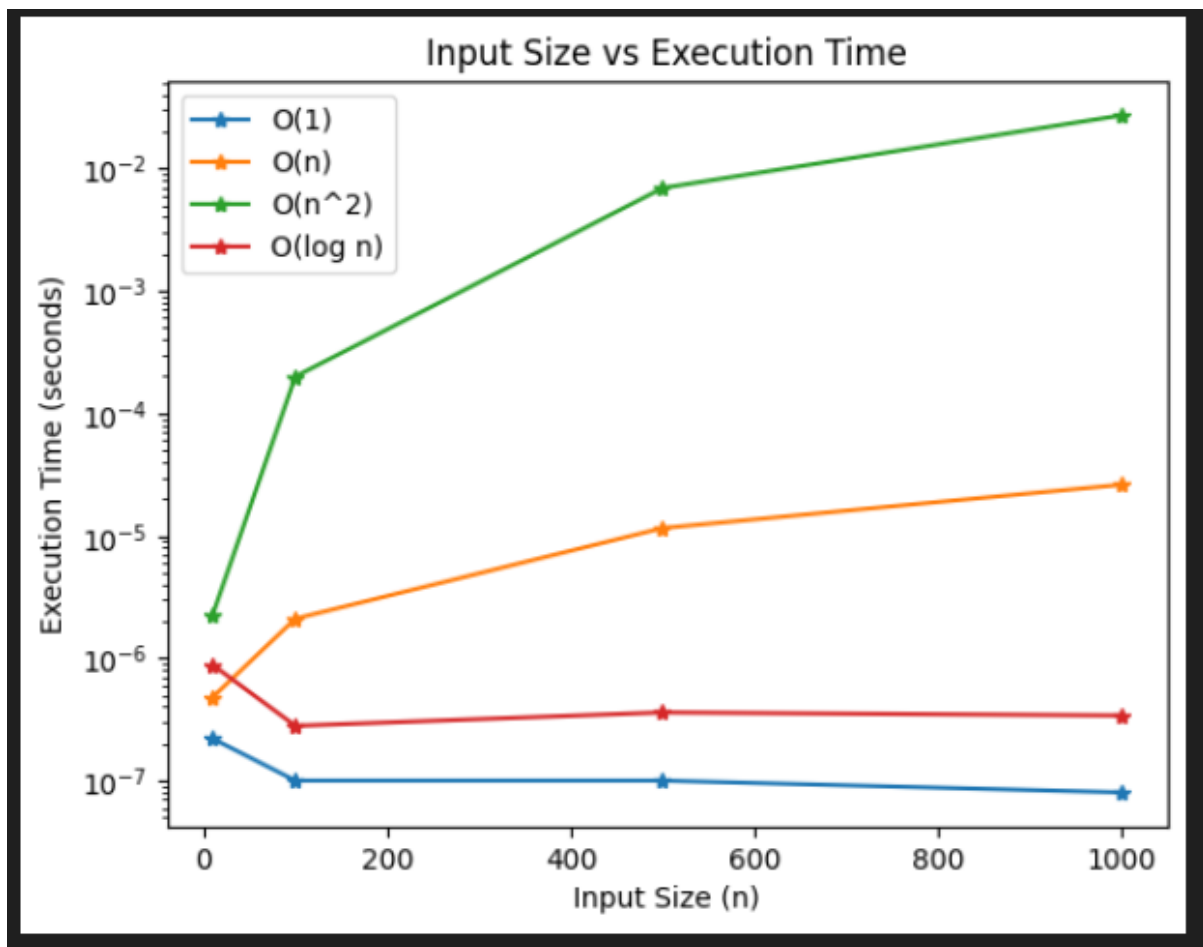
- $O(1)$  Constant
- $O(n)$  Linear
- $O(n^2)$  Quadratic
- $O(\log n)$  Logarithmic

### Output Table:

Execution Time Table:

|   | n    | $O(1)$       | $O(n)$       | $O(n^2)$ | $O(\log n)$  |
|---|------|--------------|--------------|----------|--------------|
| 0 | 10   | 3.999914e-07 | 7.599941e-07 | 0.000003 | 8.199946e-07 |
| 1 | 100  | 1.400011e-07 | 8.839997e-06 | 0.000188 | 2.600020e-07 |
| 2 | 500  | 3.200024e-07 | 2.096000e-05 | 0.006842 | 3.599911e-07 |
| 3 | 1000 | 1.200009e-07 | 7.206000e-05 | 0.023518 | 3.600027e-07 |

**Graph:**



**Observation:**

- $O(1)$  remains almost constant for all input sizes.
- $O(\log n)$  increases very slowly as  $n$  grows.
- $O(n)$  increases steadily with input size.

## Task 2: Best, Average, and Worst Case Analysis

### Aim:

To implement **Linear Search** and **Binary Search** and compare their execution time for best, average, and worst cases.

### Input Sizes

n = 100, 500, 1000, 5000, 10000

### Linear Search

- **Best Case:** target at first index
- **Average Case:** target in middle
- **Worst Case:** target not present

### Binary Search

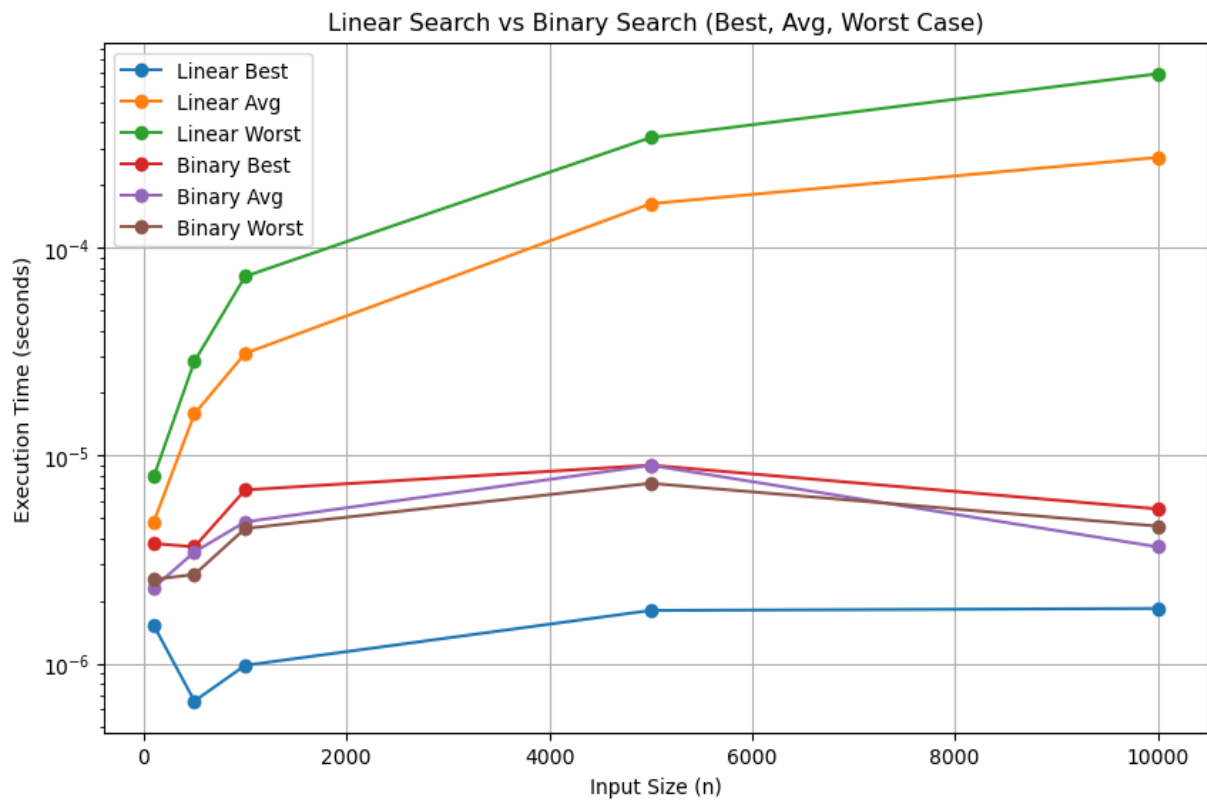
- **Best Case:** target at middle
- **Average Case:** target found after some steps
- **Worst Case:** target not present

### Output Table:

Execution Time Table:

|              | n     | Linear Best  | Linear Avg | Linear Worst | Binary Best | Binary Avg \ |
|--------------|-------|--------------|------------|--------------|-------------|--------------|
| 0            | 100   | 5.799928e-07 | 0.000001   | 0.000002     | 0.000001    | 4.399917e-07 |
| 1            | 500   | 5.800044e-07 | 0.000005   | 0.000011     | 0.000001    | 7.399940e-07 |
| 2            | 1000  | 3.599911e-07 | 0.000013   | 0.000025     | 0.000001    | 8.799951e-07 |
| 3            | 5000  | 3.399909e-07 | 0.000064   | 0.000124     | 0.000001    | 1.160009e-06 |
| 4            | 10000 | 8.200062e-07 | 0.000153   | 0.000386     | 0.000003    | 2.299994e-06 |
| Binary Worst |       |              |            |              |             |              |
| 0            | 100   | 5.800044e-07 |            |              |             |              |
| 1            | 500   | 7.200055e-07 |            |              |             |              |
| 2            | 1000  | 9.199954e-07 |            |              |             |              |
| 3            | 5000  | 1.179997e-06 |            |              |             |              |
| 4            | 10000 | 9.320001e-06 |            |              |             |              |

## Graph:



## Observation:

- Linear Search time increases significantly as n increases, especially in average and worst case.
- Binary Search stays fast even for large n because it reduces the search space each step.
- Binary search performs better than linear search for large inputs, but it requires the array to be sorted.

### Task 3: Recursion and Recurrence Validation

#### Aim

To implement recursive algorithms and compare execution time and number of function calls for recursive vs optimized methods.

#### Algorithms Implemented

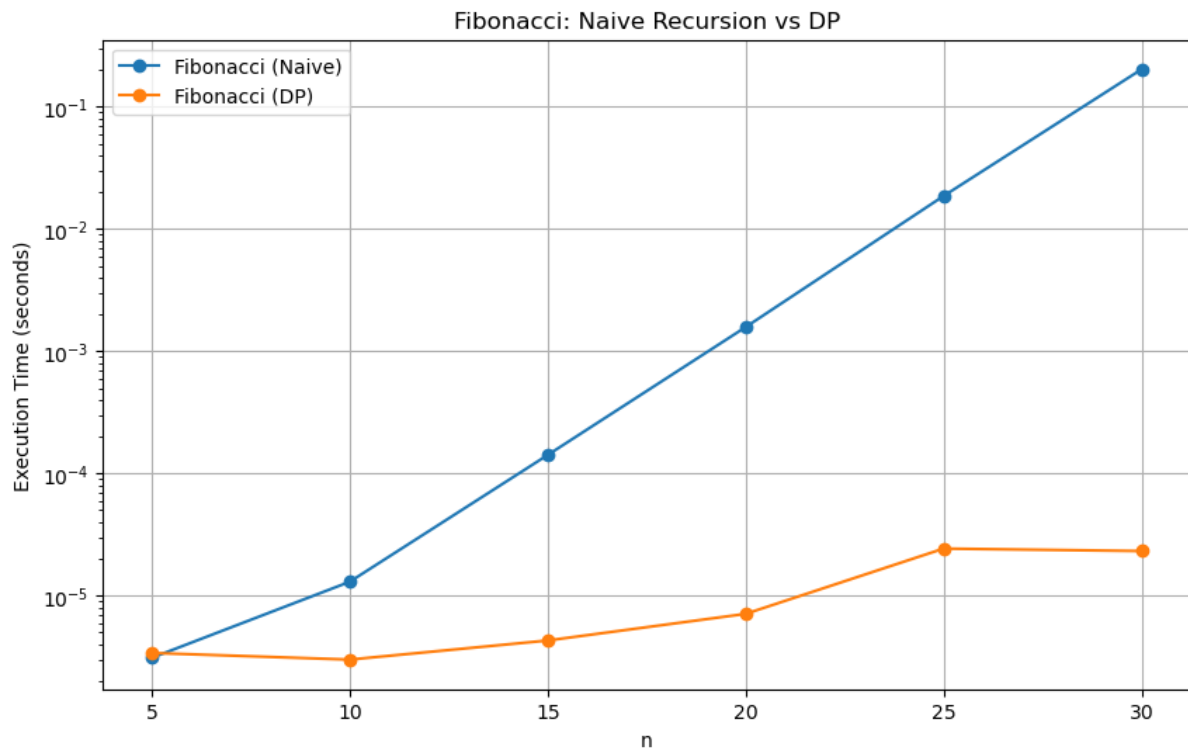
- Factorial (Recursive)
- Fibonacci (Naive Recursion)
- Fibonacci (Dynamic Programming)

#### Output Table:

Task 3 Table (Time + Calls):

|   | 0            | 1            | 2        | 3        | 4        | 5            | \ |
|---|--------------|--------------|----------|----------|----------|--------------|---|
| 0 | 100          | 5.799928e-07 | 0.000001 | 0.000002 | 0.000001 | 4.399917e-07 |   |
| 1 | 500          | 5.800044e-07 | 0.000005 | 0.000011 | 0.000001 | 7.399940e-07 |   |
| 2 | 1000         | 3.599911e-07 | 0.000013 | 0.000025 | 0.000001 | 8.799951e-07 |   |
| 3 | 5000         | 3.399909e-07 | 0.000064 | 0.000124 | 0.000001 | 1.160009e-06 |   |
| 4 | 10000        | 8.200062e-07 | 0.000153 | 0.000386 | 0.000003 | 2.299994e-06 |   |
|   |              | 6            |          |          |          |              |   |
| 0 | 5.800044e-07 |              |          |          |          |              |   |
| 1 | 7.200055e-07 |              |          |          |          |              |   |
| 2 | 9.199954e-07 |              |          |          |          |              |   |
| 3 | 1.179997e-06 |              |          |          |          |              |   |
| 4 | 9.320001e-06 |              |          |          |          |              |   |

## Graph:



## Observation:

- Factorial recursion makes linear number of calls ( $O(n)$ ).
- Naive Fibonacci makes a very large number of calls and becomes slow quickly (exponential).
- DP Fibonacci reduces calls drastically and runs much faster ( $O(n)$ ).

## Task 4: Solving Recurrences through Code

### Aim:

To implement recursive functions for given recurrences and validate their growth using execution time and recursive call count.

### Recurrences Implemented:

1.  $T(n) = T(n/2) + n$
2.  $T(n) = 2T(n/2) + n$

### Output Table:

Task 4 Table:

|    | Recurrence           | n    | Value | Calls | Time (sec)   |
|----|----------------------|------|-------|-------|--------------|
| 0  | $T(n) = T(n/2) + n$  | 2    | 3     | 2     | 1.499895e-06 |
| 1  | $T(n) = 2T(n/2) + n$ | 2    | 4     | 2     | 1.000008e-06 |
| 2  | $T(n) = T(n/2) + n$  | 4    | 7     | 3     | 7.001217e-07 |
| 3  | $T(n) = 2T(n/2) + n$ | 4    | 12    | 3     | 6.000046e-07 |
| 4  | $T(n) = T(n/2) + n$  | 8    | 15    | 4     | 4.000030e-07 |
| 5  | $T(n) = 2T(n/2) + n$ | 8    | 32    | 4     | 5.001202e-07 |
| 6  | $T(n) = T(n/2) + n$  | 16   | 31    | 5     | 4.998874e-07 |
| 7  | $T(n) = 2T(n/2) + n$ | 16   | 80    | 5     | 4.998874e-07 |
| 8  | $T(n) = T(n/2) + n$  | 32   | 63    | 6     | 6.000046e-07 |
| 9  | $T(n) = 2T(n/2) + n$ | 32   | 192   | 6     | 6.000046e-07 |
| 10 | $T(n) = T(n/2) + n$  | 64   | 127   | 7     | 6.000046e-07 |
| 11 | $T(n) = 2T(n/2) + n$ | 64   | 448   | 7     | 6.998889e-07 |
| 12 | $T(n) = T(n/2) + n$  | 128  | 255   | 8     | 7.001217e-07 |
| 13 | $T(n) = 2T(n/2) + n$ | 128  | 1024  | 8     | 8.998904e-07 |
| 14 | $T(n) = T(n/2) + n$  | 256  | 511   | 9     | 7.001217e-07 |
| 15 | $T(n) = 2T(n/2) + n$ | 256  | 2304  | 9     | 8.000061e-07 |
| 16 | $T(n) = T(n/2) + n$  | 512  | 1023  | 10    | 1.100125e-06 |
| 17 | $T(n) = 2T(n/2) + n$ | 512  | 5120  | 10    | 1.099892e-06 |
| 18 | $T(n) = T(n/2) + n$  | 1024 | 2047  | 11    | 1.200009e-06 |
| 19 | $T(n) = 2T(n/2) + n$ | 1024 | 11264 | 11    | 1.100125e-06 |

## Graph:



## Observation:

- For  $T(n) = T(n/2) + n$ , the growth is approximately  $O(n)$ .
- For  $T(n) = 2T(n/2) + n$ , the growth is approximately  $O(n \log n)$ .
- Both recurrences make around  $\log n$  recursive calls, but the second one does more work due to the multiplication by 2.

**Github:**<https://github.com/Nighthawk-208/Analysis-And-Design-Of-Algorithms->